Exact Evaluation of QOp with Cyclotomic Arithmetic

Matrix and statevector semantics, and projects for exact MPS

QPP / HQP implementation note · 16 October 2026

Abstract

QPP already expresses gate angles as rational multiples of π. We show that every well-formed operator in this syntax has a matrix over a finite cyclotomic field determined by the program. This gives exact dense matrix and direct statevector semantics without changing `QOp`, computing matrix exponentials, or introducing a new state typeclass. We specify scalar representations, evaluator algorithms, integration boundaries, and exact acceptance tests for their implementation. We then distinguish matrix product states (MPS) from their usual SVD-based canonicalization: the former admit exact cyclotomic representations, whereas the latter can require noncyclotomic numbers. Exact rank factorization provides an alternative, developed here into a four-week HQP project and an MSc thesis proposal. This is an implementation design; performance measurements and completed backend validation are future work.

1 Scope and the exactness result

We retain the current `QOp` syntax. Constructors appear in monospace; u, t denote terms. For a valid term u, write n_u for its qubit width and $[\![u]\!] \in M_{2^{n_u}}(\mathbb{C})$ for its matrix denotation. A valid term has nonnegative widths, matching operands for composition and direct sum, bijective permutations, and a *Hermitian unitary* denotation for every rotation axis. For `R u q`, set $A = [\![u]\!]$; the denotation is $\exp(-i\pi q A/2)$. `Compose u t` applies t first. Qubit zero is the most significant bit; the left tensor factor occupies the leading wires.

The initial scope is `QOp` evaluation and sequences of `Unitary` steps on an already supplied state. QPP's `Program` also admits `Measure` and `Initialize`; these are explicitly outside the first exact evaluator. In particular, `Initialize` currently performs measurement and correction, rather than merely constructing a basis vector.

Write $\zeta_N = e^{2\pi i/N}$, $K_N = \mathbb{Q}(\zeta_N)$, and $D_N = \mathbb{Z}[1/2, \zeta_N]$. Write $s \preceq t$ when s is a syntactic subterm of t, including t itself. Choose

$$N = \operatorname{lcm}(\{8\} \cup \{2b : \texttt{Phase}\ (a/b) \preceq t\} \cup \{4b : \texttt{R}\ u\ (a/b) \preceq t\}), \tag{1}$$

where each a/b is reduced with $b > 0$; subterms inside rotation axes are included. This is a sufficient field index, not necessarily the minimal conductor.

Proposition 1. *Every valid n-qubit `QOp` with index N from (1) denotes a unitary matrix in $M_{2^n}(D_N) \subset M_{2^n}(K_N)$.*

Proof. Phases are roots of unity; $i = \zeta_4$, and $1/\sqrt{2} = (\zeta_8 + \zeta_8^{-1})/2$. For a rotation, set $A = [\![u]\!]$, $I = I_{2^{n_u}}$, and $\theta = \pi q/2$. Hermitian unitarity gives $A^2 = I$, so even powers of A are I and odd powers are A. Separating the exponential series into these two parts gives

$$e^{-i\theta A} = \cos\theta\, I - i \sin\theta\, A.$$

For $q = a/b$, Euler's formula with $w = e^{i\theta} = \zeta_{4b}^{a}$ gives

$$\begin{aligned} [\![\texttt{R}\ u\ q]\!] &= c_q\, [\![\texttt{Id}\ n_u]\!] + d_q\, [\![u]\!], \\ c_q = \frac{w + w^{-1}}{2}, &\qquad d_q = \frac{w^{-1} - w}{2}. \end{aligned} \tag{2}$$

All remaining constructors use sums, products, conjugation, and insertion of zeros and ones. These preserve D_N. The same induction proves unitarity. □

2 The scalar layer: exact coefficients, unchanged angles

There is no need to replace the `Rational` parameters of `Phase` or `R`. Rationality makes the exponentiated angles roots of unity. The evaluator replaces approximate complex coefficients with a cyclotomic scalar. Cyclotomic field arithmetic and algorithms for roots of unity are well established [1, 2].

Representation and equality. For fixed N, represent an element as a rational polynomial

$$z = \sum_{j=0}^{d-1} z_j \zeta_N^j, \qquad d = \varphi(N), \qquad z_j \in \mathbb{Q},$$

reduced modulo the cyclotomic polynomial $\Phi_N(x)$. Addition is coefficientwise; multiplication is polynomial multiplication followed by reduction. Conjugation substitutes $x \mapsto x^{N-1}$ and reduces. The irreducibility of Φ_N makes coefficient equality exact equality in the field. An inverse of a nonzero element follows from the extended Euclidean algorithm in $\mathbb{Q}[x]$. Evaluation itself needs only halving; general inverses are needed later for rank factorization.

Different field indices must not be compared as raw coefficient vectors. Embed both operands into K_L, $L = \mathrm{lcm}(N, M)$, using $\zeta_N \mapsto \zeta_L^{L/N}$, and reduce there. A common field for an entire evaluation avoids repeated promotion. For an externally supplied state in K_M, enlarge the circuit field to $K_{\mathrm{lcm}(N,M)}$ before application.

A small, explicit API. Keep constructors hidden; the declarations below specify the scalar interface:

```
-- Cyclo is an opaque exact complex cyclotomic type.
data ScalarError = InvalidRootOrder | DivisionByZero

rat    :: Rational -> Cyclo
root   :: Integer -> Either ScalarError Cyclo
add, sub, mul :: Cyclo -> Cyclo -> Cyclo
neg, conj    :: Cyclo -> Cyclo
inv    :: Cyclo -> Either ScalarError Cyclo
isZero :: Cyclo -> Bool
cisPi  :: Rational -> Cyclo   -- exp(i*pi*q), via a root
```

Here `root n` is the distinguished root $e^{2\pi i/n}$, not an unspecified primitive root. Reduce integer powers modulo the root order; negative angles need no transcendental computation. The mathematical API above omits resource failures, which the evaluator must handle separately.

For the first implementation, use an adapter around an existing exact cyclotomic library [2], with tests for embeddings, conjugation, and zero detection. The polynomial description is the reference contract; it does not prescribe that library's internal representation. Keep explicit operations in the adapter: a conventional complex `abs` requires $\sqrt{z\bar{z}}$, which need not be cyclotomic. Do not invent `Floating` or `RealFloat` instances. The standard `Data.Complex` arithmetic requires `RealFloat`, making it an unsuitable numeric interface here [3].

Cost and resource limits. A dense field element occupies d rational coefficients; naive multiplication and reduction take $O(d^2)$ rational operations. Coefficient bit lengths also grow, so field-operation counts are not bit complexities. Use `Integer` for root orders, cache roots and Φ_N where the adapter permits, and check field-degree and allocation budgets before construction. Large coprime angle denominators can make d large even for a short circuit. Existing `toRational Double` angle producers may create enormous denominators: exact evaluation then means exactness for that rationalized input, not recovery of an intended analytic angle. Exceeding configured limits must return an error, never silently switch to floating point.

3 Exact matrix semantics

The matrix evaluator implements $[\![\cdot]\!]$ as the small-circuit reference. Store a matrix as checked dimensions and a flat, row-major `Data.Vector.Vector Cyclo`; use boxed storage, not BLAS/LAPACK or the current `hmatrix` element representation. A zero-qubit operator is a 1×1 matrix. The denotation of `Phase q` must retain that shape, including when used under `C` or `DirectSum`.

Term	Matrix denotation
`Id n`	I_{2^n}.
`Phase q`	$[\, e^{i\pi q} \,]$.
`X, Y, Z, H, SX`	The matrices in (3).
`R u q`	$c_q I_{2^{n_u}} + d_q [\![u]\!]$, using (2).
`C u`	$\mathrm{diag}(I_{2^{n_u}}, [\![u]\!])$.
`Permute ks`	$P_{f(j),j} = 1$ and other entries zero, where the bits of $f(j)$ are $b_{\mathtt{ks}[0]}, \ldots, b_{\mathtt{ks}[n-1]}$ for input bits $b_0, \ldots, b_{n-1}$ and $n = \lvert\mathtt{ks}\rvert$.
`Tensor u t`	$[\![u]\!] \otimes [\![t]\!]$.
`DirectSum u t`	$\mathrm{diag}([\![u]\!], [\![t]\!])$, requiring $n_u = n_t$.
`Compose u t`	$[\![u]\!]\,[\![t]\!]$, requiring $n_u = n_t$.
`Adjoint u`	$[\![u]\!]^\dagger$, by conjugate transpose.

$$
\begin{gathered}
X := [\![\mathtt{X}]\!] = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y := [\![\mathtt{Y}]\!] = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z := [\![\mathtt{Z}]\!] = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \\
H := [\![\mathtt{H}]\!] = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad \mathrm{SX} := [\![\mathtt{SX}]\!] = \frac{1}{2} \begin{pmatrix} 1+i & 1-i \\ 1-i & 1+i \end{pmatrix}.
\end{gathered} \tag{3}
$$

The rotation formula replaces `matFunc exp`; no diagonalization is needed. For accepted terms, the structural type checker supplies a sufficient proof that every rotation axis is Hermitian. Validate once, then interpret a private checked representation. A rejected proof is not a claim that the matrix is non-Hermitian. An optional later fallback may evaluate an axis exactly and check $A = A^\dagger$; it is not required for the initial evaluator.

Construction and application. Use strict loops for matrix multiplication and matrix-vector multiplication:

$$
(AB)_{ij} = \sum_k A_{ik} B_{kj}, \qquad (Av)_i = \sum_j A_{ij} v_j.
$$

Internal sums begin with `rat 0`; test zero exactly when skipping sparse contributions. Never use magnitude thresholds. The tensor indexing rule is $(A \otimes B)_{ir+k,\, js+l} = A_{ij} B_{kl}$ for B of shape $r \times s$. Check dimensions and multiplication overflow before allocating the vector.

Phase-sensitive semantics. Semantic equality $[\![u]\!] = [\![t]\!]$ is entrywise, including global phase. The derived `Eq` instance for `QOp` instead compares syntax. Writing $S = \mathrm{diag}(1, i)$ and $T = \mathrm{diag}(1, \zeta_8)$, we have $[\![\mathtt{R\ Z\ (1/2)}]\!] = e^{-i\pi/4} S$ and $[\![\mathtt{R\ Z\ (1/4)}]\!] = e^{-i\pi/8} T$. The terms `Phase (1/4) <.> R Z (1/2)` and `Phase (1/8) <.> R Z (1/4)` denote S and T, respectively; `<.>` is syntactic tensor product. A global phase becomes a relative phase under control. Likewise, $[\![\mathtt{R\ (Id\ n)\ q}]\!] = e^{-i\pi q/2} I_{2^n}$.

For $D = 2^n$, dense storage is $O(D^2)$ field elements, matrix multiplication is $O(D^3)$ field operations, and application is $O(D^2)$. This backend is an exact oracle, not a claim of efficient general quantum simulation.

4 Direct exact statevector semantics

Store 2^n cyclotomic amplitudes in a boxed vector. A basis constructor places one at the selected bit-string index and zero elsewhere; the empty bit string gives the one-dimensional state (1). More generally, accept arbitrary vectors with a checked power-of-two length: intermediate control branches are not normalized, even when the full input is. No vector operation calls `normalize`.

Let $F_u : K^{2^{n_u}} \to K^{2^{n_u}}$ be the direct evaluator for a checked term u, with K containing both operator and input coefficients. Identity leaves the vector unchanged; `Phase` q maps (z) to $(e^{i\pi q} z)$ on zero qubits. The one-qubit primitives act on amplitude pairs. The structural cases are:

$$F_{\texttt{Compose}\ u\ t}(v) = F_u(F_t(v)), \tag{4}$$

$$F_{\texttt{R}\ u\ q}(v) = c_q\, v + d_q\, F_u(v), \tag{5}$$

$$F_{\texttt{DirectSum}\ u\ t}(v_0 \parallel v_1) = F_u(v_0) \parallel F_t(v_1), \tag{6}$$

$$F_{\texttt{C}\ u}(v_0 \parallel v_1) = v_0 \parallel F_u(v_1). \tag{7}$$

Here c_q, d_q are from (2); $\parallel$ is vector concatenation. The split is by the leading selector bit. Equation (5) recursively evaluates the complete axis, not only a Pauli string. It therefore handles `R H q`, controlled Hermitian axes, and all other axes accepted by the checker. Preserve v until both contributions have been formed; in-place overwrite would make the recurrence incorrect.

Tensor products and strides. For `Tensor u t`, set $A = [\![u]\!]$, $B = [\![t]\!]$, and $D_x = 2^{n_x}$. Reshape the vector as $V_{ij} = v_{iD_t+j}$, of shape $D_u \times D_t$. Apply F_t to each contiguous row, then F_u to each column, gathered with stride D_t:

$$W_{ij} = \sum_l B_{jl} V_{il}, \qquad V'_{ij} = \sum_k A_{ik} W_{kj}.$$

Flatten V' in the same order. Initially, explicit gather/scatter loops are preferable to a complex view abstraction. Later, compile windows and strides once to avoid repeated index work. This implements tensor action without forming A, B, or $A \otimes B$ as dense matrices.

Permutations and adjoints. For `Permute ks`, compute the same bit permutation f as in Section 3 and scatter $v'_{f(j)} = v_j$; a gather implementation must use f^{-1}. Compile a term together with a dagger flag: reverse the order of composition, negate phase and rotation angles, invert permutations, and recurse through tensors, control, and direct sums. The Hermitian primitive matrices are self-adjoint; $\mathrm{SX}^\dagger$ uses the conjugate-transpose 2×2 table. For a valid rotation, $[\![\texttt{R}\ u\ q]\!]^\dagger = [\![\texttt{R}\ u\ (-q)]\!]$: the axis term remains u.

Proposition 2. *For every checked term u and compatible vector v, the direct evaluator satisfies $F_u(v) = [\![u]\!]\, v$ exactly. If v has cyclotomic entries, so does the result; if $v^\dagger v = 1$, then $F_u(v)^\dagger F_u(v) = 1$.*

Proof. Induct over the checked term. The tensor index equations give the Kronecker-product action; the other cases follow from their matrix definitions. Unitarity gives preservation of the inner product. □

For G local gate applications, evaluation uses $O(G2^n)$ field operations and $O(2^n)$ storage with reusable buffers. A general AST costs $O(s2^n)$ for expanded tree size s, up to indexing overhead. Naive recursive rotations can retain $O(h2^n)$ live elements at nesting depth h; $O(2^n)$ working storage is not a blanket bound for arbitrary ASTs.

5 Implementation plan and acceptance criteria

Add exact backends alongside the numerical ones. Keep `RealT`, `ComplexT`, `HilbertSpace`, and `QOp` unchanged. Cyclotomic complex scalars are used directly, without constructing a fake `Complex RealT` or an instance requiring `Floating`. The following module names and signatures are proposed interfaces, not claims about modules already present in the repository.

Proposed module	Responsibility
`QPP.Exact.Scalar`	Opaque cyclotomic adapter, exact operations, root construction, optional approximate export.
`QPP.Exact.Checked`	Validate widths and axes; collect field index; enforce resource limits; compile an internal checked tree.
`QPP.Semantics.Exact.Matrix`	Boxed dense operator oracle and matrix-vector application.
`QPP.Semantics.Exact.Statevector`	Direct application, basis states, inner products, and squared norms.

```
-- Each backend exports its own opaque OpT and StateT.
evalOp   :: Limits -> QOp -> Either ExactError OpT
ket      :: Limits -> [Bool] -> Either ExactError StateT
apply    :: OpT -> StateT -> Either ExactError StateT
evalProg :: Limits -> Program -> StateT
         -> Either ExactError StateT
```

`OpT` stores a matrix in the Matrix backend and a checked application plan in Statevector; both represent $[\![u]\!]$. Their `StateT`s store vectors, without asserting unit norm. `evalProg` validates every step and its width before execution, rejects `Measure`/`Initialize` with the offending step index, and applies unitary steps in list order. The empty program returns its input. Error cases include existing `TypeError`s, state/operator width mismatch, index overflow, and a resource limit; pure evaluation must not rely on unchecked array exceptions. `OpT` retains its limits; `apply` checks the joint field of operator and input before promotion. Approximate export is a separate, one-way observation, never used for equality or pivot selection.

Build order for the first implementation session. (1) Establish scalar laws and exact phase/rotation coefficients. (2) Implement matrix construction for every constructor and the checked entry point. (3) Implement direct statevector action, with matrix application as the independent oracle. (4) Add program preflight, examples, and bounded resource diagnostics. Correctness is the initial completion criterion; conductor minimization, optimized memory schedules, and specialized arithmetic remain later improvements.

Tests that decide completion.

1. Scalar tests: roots satisfy $z^n = 1$; mixed-field embeddings agree; conjugation is involutive; exact cancellation gives zero; nonzero division works and zero division is rejected. Include odd conductors and negative angles, not just powers of two.
2. Gate tests: $H^2 = X^2 = Y^2 = Z^2 = I_2$ and $\mathrm{SX}^2 = X$. For $U_q = [\![\mathtt{R}\ u\ q]\!]$, check $U_q U_{-q} = I_{2^{n_u}}$ and $U_q U_r = U_{q+r}$. Use axis terms `H`, `C X`, `Id n`, and a conjugated Pauli term.
3. Differential tests: for bounded well-typed random ASTs, every column from direct evaluation of a basis vector equals the dense matrix column. Check $[\![u]\!]^\dagger [\![u]\!] = I_{2^{n_u}}$ exactly. Include zero-qubit phases, nested control, unequal tensor widths, direct sums, and non-self-inverse permutations.
4. Analytic tests: Bell preparation, phase-sensitive S/T gates, and small QFTs against $\zeta_{2^n}^{jk}/\sqrt{2^n}$ with the repository's wire ordering. Bound denominators and AST size in generators; preserve typing while shrinking.
5. Rejection tests: malformed axes, incompatible widths, invalid permutations, unsupported program steps, and resource failures.

6 Exact MPS: replace SVD with field rank factorization

An MPS represents amplitudes as $v_{b_1 \ldots b_n} = A^{[1]}_{b_1} \cdots A^{[n]}_{b_n}$, with scalar boundary dimensions; site j corresponds to QPP wire $j-1$. Tensor contraction and local gates preserve a common coefficient field. The obstruction is the orthonormal gauge usually constructed by SVD [5], not the MPS representation itself.

A concrete obstruction. Let

$$M = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 2 \end{pmatrix}, \qquad \det(xI - M^T M) = x^3 - 8x^2 + 15x - 4.$$

The cubic has no rational root and discriminant $916 = 4 \cdot 229$. It is irreducible with Galois group S_3, so the field generated by any root is a non-Galois cubic extension. It cannot be a subfield of a cyclotomic field, whose Galois group is abelian [1]. Thus even the *squared* singular values can leave cyclotomics. Since $\|M\|_F^2 = 8$, padding $M/\sqrt{8}$ to 4×4 gives a normalized four-qubit state with noncyclotomic Schmidt coefficients. It is `QOp`-preparable: a Hadamard followed by two Hadamards controlled on one branch gives amplitudes $(2, 1, 1, 1, 1)/\sqrt{8}$; a reversible basis permutation places them as shown. QPP's existing `Exact` MPS setting still calls a floating-point `compactSVD`; it is not an algebraic exactness mode.

Exact factorization. For $T \in K^{m \times n}$ over a cyclotomic field K, with rank r, select r independent columns J and independent rows I of $B = T_{:,J}$. Then

$$T = BC, \qquad B = T_{:,J}, \qquad C = (B_{I,:})^{-1} T_{I,:}. \tag{8}$$

Implement the inverse action by elimination rather than forming an inverse; treat the zero matrix separately. Exact zero tests select pivots. All factors lie in the same field K, although they may leave the smaller dyadic ring D_N. The rank is unchanged on extension from K to $\mathbb{C}$, since nonzero minors remain nonzero. Successive rank factorizations of a dense tensor therefore give an exact MPS with the usual minimal cut ranks [6]. This existence argument does not require computing Schmidt coefficients.

Two-site update. Contract neighboring cores, apply a two-qubit gate matrix U, and reshape (physical indices $s, t, a, b \in \{0, 1\}$):

$$T_{(\alpha,s),(t,\beta)} = \sum_{a,b,\gamma} U_{st,ab} A^{[j]}_a(\alpha, \gamma) A^{[j+1]}_b(\gamma, \beta).$$

Factor $T = BC$ by (8), reshape B and C into new cores, and absorb all coefficients without normalization. One-site gates act directly on the physical index. Adjacent SWAPs provide a simple initial route for nonlocal gates. A local factorization removes dependence in that local unfolding; it does *not* guarantee globally minimal bonds if the surrounding block bases are redundant. Global compression requires left/right rank-revealing sweeps and their own invariants.

Contractions without a canonical center. Compute squared norms and inner products through environments, rather than from one privileged core. For a squared norm, propagate $E_0 = [1]$ and $E_j = \sum_s (A^{[j]}_s)^\dagger E_{j-1} A^{[j]}_s$. The final scalar is $\|v\|^2$ exactly. A local two-site update costs $O(\chi^3)$ field operations for uniform bond bound χ; core storage is $O(n\chi^2)$ field elements. Neither bound controls coefficient height or field degree. Exact rank reduction discards only algebraic redundancy: there is no tolerance, small-singular-value truncation, or guarantee that the bond dimensions remain small.

7 HQP project: an exact MPS interpreter in four weeks

Question and starting point. Can rank factorization replace SVD in a useful, verifiably exact MPS simulator for local quantum circuits? The project assumes the instructor supplies the tested cyclotomic adapter, exact Matrix and Statevector oracles from Sections 2–5, and a small circuit corpus. Building number-field arithmetic is not part of this month's work. Expected prerequisites are Haskell, tensor indexing, Gaussian elimination, and the circuit model.

Required scope. Implement opaque MPS cores over the supplied field with checked bond dimensions; basis-state construction; exact contraction; one-site and adjacent two-site unitary updates; and rank factorization. Support X, Y, Z, H, SX, rational rotations about X, Y, Z, global phase, CNOT, controlled rational phase, and adjacent SWAP. Use a small positioned-gate IR, supplied or implemented with explicit width checks. A narrow `QOp` front end may flatten composition, tensor padding, and this whitelist of local gates. General nested control, arbitrary `DirectSum`, and composite rotation axes must report unsupported input rather than materialize a full matrix secretly.

Week	Work	Demonstrable result
1	Understand the scalar API; implement exact rank factorization, including rectangular, rank-deficient, and zero matrices.	Tests verify $T = BC$, full factor rank, and deterministic pivot behavior.
2	MPS cores, basis states, contraction, and one-/two-site updates. Compare amplitudes after every gate.	Bell and GHZ preparation and inverse circuits agree exactly with the statevector oracle.
3	Gate-IR integration, adjacent SWAPs, environment contraction, and bounded randomized tests.	The supported fragment has explicit errors and exact differential tests; squared norms are computed without SVD assumptions.
4	Reproducible experiments, code cleanup, report, and demonstration.	Explain measured costs using bonds, field degree, coefficient growth, and circuit structure.

Acceptance criteria and deliverables. Deliver an integrated Haskell module, tests, a reproducible benchmark driver, and a concise report (suggested 6–10 pages, subject to course requirements). Tests must include entanglement creation and removal, rank deficiency, exact cancellations, non-dyadic rational angles, phase-sensitive controlled gates, and SWAP ordering. For bounded circuits on up to eight qubits, compare all amplitudes against the exact statevector oracle, including intermediate states. Check squared norm one and circuit/inverse round trips exactly.

Report wall time, peak memory when available, maximum bond dimension, field index/degree, and coefficient bit length when exposed by the adapter. Sweep circuit depth and angle denominators separately. Use product circuits, GHZ chains, and shallow entangling circuits; a compiled small QFT is a stretch example. Report failed resource limits as results, not as missing observations. There is no required speedup over dense evaluation and no prescribed large-qubit target.

Scope control. The minimum successful project has correct local rank reduction; it does not need globally minimal bonds. General `QOp` semantics, measurement, SVD, approximation, GPU work, and a new cyclotomic library are excluded. If time remains, add a justified rank-compression sweep or nonadjacent-gate routing, rather than broadening several interfaces at once. This leaves a substantial but bounded project even when exact coefficient growth limits the achievable circuit sizes.

8 MSc thesis: exact tensor-network semantics and coefficient growth

Research aim. Determine when exact MPS/MPO evaluation is an effective semantics for QPP, and which representation and compression choices control its cost. A six-month full-time schedule is an indicative planning assumption, to be adjusted to the actual thesis arrangement. The contribution must extend beyond implementing a larger gate set: it should connect proved invariants to measured arithmetic behavior.

Three research questions.

1. Can the complete checked `QOp` fragment be interpreted compositionally using exact MPS/MPO arithmetic, with a correctness argument against the dense semantics and no hidden dense fallback?
2. Which pivot policies, scalar representations, and compression schedules best control coefficient height and work at a given exact bond rank? Does smaller bond dimension actually imply faster exact evaluation on the workloads studied?
3. How do entanglement, angle denominators, and program structure separately constrain performance, and where is exact evaluation useful as a compiler-validation oracle?

Core technical work. Define field-valued MPS/MPO representations with explicit gauge and wire-order invariants. Extend local evaluation to general operators using MPO arithmetic or direct branching. With $P_b = |b\rangle\langle b|$,

$$[\![\,\mathtt{C}\ u\,]\!] = P_0 \otimes I_{2^{n_u}} + P_1 \otimes [\![u]\!],$$
$$[\![\,\mathtt{DirectSum}\ u\ t\,]\!] = P_0 \otimes [\![u]\!] + P_1 \otimes [\![t]\!] \quad (n_u = n_t).$$

Rotations use (5). Exact MPS addition may concatenate or block-diagonalize bonds, followed by rank-based compression. Handle zero branches explicitly.

Develop and justify global rank-revealing sweeps. Distinguish the existence of minimal-rank MPS from the correctness of a particular local compression algorithm. Compare deterministic first-nonzero pivots with coefficient-size heuristics, eager with bond-size-triggered exact compression, and a common circuit field with dynamic subfield representations. Choose two of these comparisons as the core experimental study; fraction-free or modular elimination is an optional extension, not a simultaneous requirement.

Period	Milestone
Month 1	Literature review, reproduce the local prototype, define semantics and experiment protocol.
Months 2–3	General compositional evaluator and rank-sweep invariants; exact agreement tests; complete a baseline implementation.
Month 4	Implement the selected arithmetic/compression variants; instrument field operations, bonds, and coefficient sizes.
Month 5	Controlled experiments and compiler-rewrite case studies; explain successes and failures.
Month 6	Consolidate proofs and artifacts, write the thesis, and prepare a reproducible release and presentation.

Evaluation and success. Use exact Matrix/Statevector as correctness oracles at small sizes, uncompressed exact MPS as a compression baseline where feasible, and the existing floating MPS only as a performance/accuracy comparison. Vary bond growth and conductor growth independently; include low-entanglement circuits, QFTs, random Clifford+T and mixed-denominator circuits, plus phase-sensitive rewrite regressions. Publish seeds, limits, build information, raw results, and all timeouts. Success means proved preservation for the supported semantics, reproducible comparative evidence, and an explanation of the limits; universal speedup is neither expected nor required. Measurement with unnormalized branches, certified approximate export, or parallel arithmetic are separate extensions after the core questions are answered.

9 Boundaries, related work, and the implementation decision

Cyclotomic arithmetic supports exact amplitudes, matrix entries, inner products, squared norms, and Born probabilities. It does not support arbitrary normalization or SVD. For example, the cyclotomic vector $(1, 2 + \sqrt{2})$ has squared norm $7 + 4\sqrt{2}$, whose square root is not cyclotomic. Its minimal polynomial is $x^4 - 14x^2 + 17$; over $\mathbb{Q}(\sqrt{2})$ the conjugate radicands $7 \pm 4\sqrt{2}$ have product 17, and their square roots do not belong to the same quadratic extension. Consequently the resulting real quartic field is not Galois.

These boundaries should be reflected in the API, not hidden behind partial implementations of `sqrt`. Nevertheless, measurement is not intrinsically incompatible with exact arithmetic. Unnormalized projected vectors remain cyclotomic, and their probabilities are ratios of exact squared norms. For an outcome projector P and density matrix ρ with entries in the same cyclotomic field, conditioning gives $P\rho P^\dagger / \mathrm{tr}(P\rho P^\dagger)$ for a nonzero probability, which also stays in the field. Sampling introduces an additional ordering/randomness problem. None of these extensions is needed to implement exact `QOp` evaluation now.

The arithmetic foundation is classical cyclotomic field theory [1]; Walck's Haskell library supplies a reusable implementation [2]. Exact Clifford+T synthesis already connects quantum circuits with the dyadic cyclotomic ring [4]. Our direction is evaluation of an existing language, not a new synthesis theorem. The familiar MPS and tensor-train constructions [5, 6] explain the role of unfolding ranks and canonical gauges. MPO arithmetic and elimination of linear dependence also have established precedents [7]. The proposed student work studies these ideas specifically with exact cyclotomic arithmetic, QPP semantics, and reproducible coefficient-growth experiments; it does not claim novelty for tensor contraction or rank factorization.

The immediate implementation decision is therefore narrow: retain rational angles, validate the existing syntax, introduce an opaque exact scalar layer, and implement independent boxed Matrix and Statevector backends. Their exact agreement establishes the reference semantics and the test infrastructure for the two MPS projects. The scientific distinction to preserve is between the field required by a quantum state and the larger fields that a chosen factorization algorithm may introduce.

References

[1] J. S. Milne. *Fields and Galois Theory*, version 5.10, 2022. Sections on Galois correspondence and cyclotomic extensions. https://www.jmilne.org/math/CourseNotes/FT.pdf.

[2] S. N. Walck. *cyclotomic: A subfield of the complex numbers for exact calculation*, Haskell package, version 1.1.2, 2023. https://hackage.haskell.org/package/cyclotomic-1.1.2.

[3] GHC libraries. *Data.Complex*, `base-4.21.0.0` documentation. https://hackage.haskell.org/package/base-4.21.0.0/docs/Data-Complex.html.

[4] B. Giles and P. Selinger. Exact synthesis of multiqubit Clifford+T circuits. *Physical Review A* 87, 032332, 2013. https://doi.org/10.1103/PhysRevA.87.032332.

[5] U. Schollwöck. The density-matrix renormalization group in the age of matrix product states. *Annals of Physics* 326, 96–192, 2011. https://doi.org/10.1016/j.aop.2010.09.012.

[6] I. V. Oseledets. Tensor-train decomposition. *SIAM Journal on Scientific Computing* 33(5), 2295–2317, 2011. https://doi.org/10.1137/090752286.

[7] C. Hubig, I. P. McCulloch, and U. Schollwöck. Generic construction of efficient matrix product operators. *Physical Review B* 95, 035129, 2017. https://doi.org/10.1103/PhysRevB.95.035129.